

# Lección 10: Strategic Coverage

---

## Coverage Honesto (100/80/0)

---

### Agenda

---

- Coverage Ciego vs Estratégico
  - Sistema 100/80/0
  - Configuración Práctica
  - Casos de Uso
  - Anti-Patterns
- 

### El Problema

---

#### ❌ Coverage Ciego:

"Necesitamos 90% de coverage en toda la base de código"



Resultado:

- utils/formatDate.ts: 100% ✅ (no crítico)
- types/interfaces.ts: 95% ✅ (desperdicio)
- utils/moneyCalculations.ts: 60% ❌ (¡CRÍTICO!)



Coverage alto en código no-crítico



Coverage bajo en código que puede quebrar empresa

---

#### ✅ Coverage Estratégico

---

Sistema 100/80/0:

- 🎯 CORE (100%): Business logic crítico
  - moneyCalculations.ts: 100% ✓
  - cartOperations.ts: 100% ✓
- 🔧 IMPORTANT (80%): Features visibles al usuario
  - ShoppingCart.tsx: 85% ✓
  - PaymentForm.tsx: 87% ✓
- 🏗️ INFRASTRUCTURE (0%): Auto-validable
  - types/interfaces.ts: 0% ✓ (TypeScript valida)
  - config/constants.ts: 0% ✓ (estático)

---

## Filosofía: Coverage ≠ Quality

---

### La Verdad Incómoda:

Startup que quebró:

- 📊 Coverage general: 94% ✓
- 📊 Todos los checks de CI: PASANDO ✓
- 📊 Calidad de código: Grado A ✓

💣 Producción:

- Procesamiento de pagos: ROTO
- Cálculos de dinero: INCORRECTOS
- Datos de clientes: PERDIDOS

### Causa Raíz:

🔍 Análisis:

- Coverage alto en tests que nunca fallan
- Coverage cero en 3 funciones críticas

**94% coverage inútil si el 6% puede quebrar el negocio**

---

## Pregunta Clave

---

¿Qué código puede quebrar el negocio?

 **CORE (100%):** ¿Si falla, perdemos dinero directamente?

- Cálculos de dinero 
- Procesamiento de pagos 
- Validación de datos 

 **IMPORTANT (80%):** ¿Si falla, usuarios se frustran?

- Componentes de UI 
- Interacciones de usuario 

 **INFRASTRUCTURE (0%):** ¿Se valida automáticamente?

- Tipos de TypeScript 
- Constantes 

---

## CORE TIER: 100% Requerido

---

### Criterios:

- Manipula dinero
- Procesa datos críticos
- Lógica de negocio compleja
- Funciones de seguridad

### Ejemplo:

```
// utils/cart-operations.ts - 100% REQUERIDO
export function addItemToCart(cart, item, quantity) {
  // 🚨 CRÍTICO: Cálculos de dinero
  const totalPrice = item.price * quantity

  // Cualquier bug = pérdida de ingresos
}

// TODOS los casos extremos deben ser testeados
```

---

## IMPORTANT TIER: 80% Requerido

---

### Criterios:

- Componentes de UI visibles
- Interacciones de usuario
- Validación de formularios
- Navegación

### Ejemplo:

```
// components/ShoppingCart.tsx - 80% REQUERIDO
export function ShoppingCart() {
  return (
    <div>
      {items.map(item => (
        <CartItem key={item.id} item={item} />
      ))}
    </div>
  )
  // Enfocarse en flujos principales del usuario
  // Omitir casos extremos raros
}
```

---

## INFRASTRUCTURE TIER: 0% Estratégico

---

### Criterios:

- Interfaces de TypeScript
- Objetos de configuración
- Constantes
- Utilidades estáticas

### Ejemplo:

```
// types/cart.ts - 0% ESTRATÉGICO
export interface Cart {
  items: CartItem[]
}

// ¿Por qué 0%?
// - TypeScript valida la estructura
```

```
// - Sin lógica que testear  
// - Testear = pérdida de tiempo
```

---

## Configuración vitest.config.ts

---

```
export default defineConfig({  
  test: {  
    coverage: {  
      thresholds: {  
        // 🎯 CORE: 100%  
        'src/utils/cart-operations.ts': {  
          statements: 100,  
          branches: 100,  
          functions: 100,  
          lines: 100,  
        },  
  
        // 🛠 IMPORTANT: 80%  
        'src/components/*.tsx': {  
          statements: 80,  
          functions: 90,  
        },  
  
        // 🏗 INFRASTRUCTURE: 0%  
        'src/types/*.ts': {  
          statements: 0,  
        },  
      },  
    },  
  },  
})
```

---

## Caso 1: E-commerce Startup

---

Antes (Coverage Ciego):

```
📊 General: 95% ✅  
├─ types/product.ts: 100% (desperdiciado)  
├─ config/theme.ts: 98% (desperdiciado)  
├─ utils/pricing.ts: 60% ❌ (CRÍTICO FALTANTE)  
└─ utils/taxes.ts: 45% ❌ (CRÍTICO FALTANTE)
```

💣 Cálculos de impuestos incorrectos → quejas | 💣 Bugs de precios → pérdidas

### Después (100/80/0):

🎯 CORE (100%):  
└─ utils/pricing.ts: 100% ✅ (dinero seguro)  
└─ utils/taxes.ts: 100% ✅ (correcto)

🔧 IMPORTANT (80%):  
└─ components/ProductCard.tsx: 85% ✅

🏗️ INFRASTRUCTURE (0%):  
└─ types/product.ts: 0% ✅

🎯 Resultado: Cero bugs de dinero en 6 meses

---

## Caso 2: Migración

---

### Estrategia usando 100/80/0:

Fase 1: Migrar INFRASTRUCTURE (0%)

- Sin tests que mantener
- Migración rápida
- TypeScript detecta roturas

Fase 2: Migrar IMPORTANT (80%)

- Mantener cobertura de tests
- Funcionalidad visible al usuario

Fase 3: Migrar CORE (100%)

- Suite comprehensiva
- Lógica de negocio protegida

Resultado: Migración exitosa, cero bugs

---

## Anti-Patterns

---

❌ Coverage Inflation (inflación de cobertura):

```
// MAL: Test que no agrega valor
describe('ProductInterface', () => {
  it('debería tener propiedades', () => {
    const product: ProductItem = {
      id: 'test',
      name: 'Test',
    }
    expect(product.id).toBe('test') // ¡Inútil!
  })
})

// BIEN: Omitir, TypeScript valida
```

## ❌ Mock Everything (mockear todo):

```
// MAL: Oculta problemas reales
jest.mock('../utils/pricing', () => ({
  calculatePrice: jest.fn(() => 10.99), // ¡Oculta bugs!
}))

// BIEN: Testear funciones reales
```

---

# Métricas de Éxito

---

## Antes:

- General: 92%  (falsa seguridad)
- Tiempo de testing: 40% del desarrollo
- Bugs en producción: 12/mes
- Mantenimiento: 8 horas/semana

## Después (100/80/0):

- CORE: 100%  (seguridad real)
- Tiempo de testing: 25% (enfocado)
- Bugs en producción: 2/mes
- Mantenimiento: 3 horas/semana

**Mejora:** 80% menos bugs, 40% menos mantenimiento

---

# Ejercicio 1: Clasificar Función por Tier

---

## Prompt:

Actúa como un arquitecto de software aplicando Strategic Coverage (100/80/0

CONTEXT0: Strategic Coverage sistema 100/80/0 clasifica código por riesgo d negocio, NO por % general. Pregunta clave: "¿Qué código puede quebrar el negocio?". CORE TIER (100%): si falla → pérdida de dinero directa (cálculos de dinero, procesamiento de pagos, validación de datos). IMPORTANT TIER (80 si falla → usuarios frustrados (componentes UI, interacciones). INFRASTRUCT TIER (0%): TypeScript valida automáticamente (interfaces, constantes). Cove ≠ Quality: 94% coverage inútil si el 6% crítico falla.

TAREA: Clasifica calculateShippingCost en tier apropiado y justifica.

FUNCIÓN A ANALIZAR:

```
function calculateShippingCost(weight: number, zone: string): number {  
  if (weight <= 0) return 0  
  if (zone === 'local') return weight * 0.5  
  if (zone === 'national') return weight * 1.0  
  return weight * 2.0  
}
```

CRITERIOS DE CLASIFICACIÓN:

1. ¿Maneja dinero o datos críticos? → CORE (100%)
2. ¿Usuario interactúa visiblemente? → IMPORTANT (80%)
3. ¿Solo estructura/configuración? → INFRASTRUCTURE (0%)

ANÁLISIS REQUERIDO:

- Pregunta 1: ¿La función manipula dinero? (SÍ/NO + razón)
- Pregunta 2: ¿Un bug causa pérdida de ingresos? (SÍ/NO + razón)
- Pregunta 3: ¿Es crítico para el negocio? (SÍ/NO + razón)
- Clasificación: CORE/IMPORTANT/INFRASTRUCTURE
- Target coverage: 100%/80%/0%
- Justificación: 2-3 oraciones explicando clasificación

OUTPUT FORMAT:

Análisis de calculateShippingCost:

- Maneja dinero: [SÍ/NO] - [razón]
- Bug causa pérdida: [SÍ/NO] - [razón]
- Crítico para negocio: [SÍ/NO] - [razón]

Clasificación: [TIER]  
Target coverage: [%]  
Justificación: [explicación]

VALIDACIÓN: Clasificación debe ser CORE (100%) porque manipula costo de env

**Aprende:** Business risk classification (clasificación por riesgo de negocio) guía

coverage strategy

---

## Ejercicio 2: Configurar Coverage Threshold

---

**Prompt:**

Actúa como un ingeniero de calidad configurando coverage thresholds estraté

CONTEXT0: Coverage thresholds en vitest.config.ts fuerzan % mínimo en archi  
específicos. Sistema 100/80/0 requiere thresholds diferentes por tier: CORE  
(100% statements/branches/functions/lines), IMPORTANT (80% statements, 90%  
functions), INFRASTRUCTURE (0% excluido). Thresholds previenen merge de cód  
sin coverage adecuado: CI falla si coverage < threshold. 4 métricas de cove  
statements (líneas ejecutadas), branches (caminos if/else), functions (func  
llamadas), lines (líneas físicas).

TAREA: Configura threshold 100% para calculateShippingCost (CORE tier).

ARCHIVO A MODIFICAR:

- vitest.config.ts
- Sección: test.coverage.thresholds

THRESHOLD CONFIGURATION:

- Pattern: 'src/utils/shipping.ts'
- Tier: CORE (100%)
- Métricas requeridas (todas 100%):
  - statements: 100
  - branches: 100
  - functions: 100
  - lines: 100

ESTRUCTURA:

```
export default defineConfig({
  test: {
    coverage: {
      thresholds: {
        // 🎯 CORE: 100% - [razón: cálculos de dinero]
        "src/utils/shipping.ts": {
          // todas las métricas en 100
        },
      },
    },
  },
})
```

#### IMPLEMENTACIÓN:

1. Abrir vitest.config.ts
2. Localizar sección test.coverage.thresholds
3. Agregar entrada para 'src/utils/shipping.ts'
4. Configurar 4 métricas en 100%
5. Agregar comentario explicando por qué CORE tier

#### VALIDACIÓN:

- pnpm run test:coverage → debe FALLAR si coverage < 100% ❌
- Si falla = configuración correcta (protege código crítico)

**Aprende:** Per-file thresholds enforcement protege código crítico automáticamente en CI

---

## Puntos Clave

---

1. **Coverage ≠ Quality (calidad):** 94% puede ser inútil
2. **Strategic Tiers (niveles estratégicos):** 100/80/0 por riesgo de negocio
3. **CORE (100%):** Funciones de dinero/críticas
4. **IMPORTANT (80%):** Features visibles al usuario (user-facing)
5. **INFRASTRUCTURE (0%):** TypeScript valida
6. **Focus Resources (enfocar recursos):** Testing donde importa
7. **Measure Success (medir éxito):** Por bugs en producción, no por %